

LAB#08

INTRODUCTION TO PROLOG, OPERATORS AND BUILT-IN PREDICATES OF PROLOG

Familiarize students with prolog programming, rules and facts. Also exploring operators and built-in predicates of prolog.

I. Introduction to Prolog

There are two types of programming languages: Procedural and Declarative. Procedural languages are used to perform numerical computations like: Computing area of a circle, finding percentage marks of a student, finding the person with greater age. All these problems involve some sort of numerical computations (formulae). Some of the procedural programming languages are: C/C++, JAVA, C#, BASIC FORTRAN etc. Whereas declarative languages are used to perform non-numerical computations like: who is the mother of John? Finding all the students who like AI? Are Shahid and Saeed brothers? Now all these problems involve non-numeric computations there is no formula to find out the solutions. These types of problems cannot be solved by traditional procedural languages. Some of the declarative programming languages are: PROLOG (Programming in Logic), LISP (List Processing) and ML (Meta Language).

PROLOG stands for **PRO**gramming in **LOG**ic. Prolog is a high-level declarative programming language for symbolic and non-numeric computation. Prolog is a logic language that is particularly suited to programs that involve symbolic or non-numeric computation. For this reason, it is a frequently used language in Artificial Intelligence where manipulation of symbols and inference about them is a common task.

Prolog consists of a series of rules and facts. A program is run by presenting some query and seeing if this can be proved against these known rules and facts.

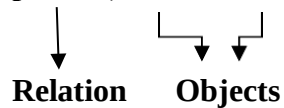
II. Basic elements of Prolog (Facts, Rules and Questions)

Prolog programs are composed of declarative sentences to be asserted. These sentences are called predicates. These predicates are of two types: Facts and Rules.

a) Facts:

PROLOG is especially well suited for solving problems that involve objects and relations between objects. An example below is a family relation. The fact that Ahmed is a parent of Ali can be written in Prolog as:

parent("ahmed", "ali"). Read as: Ahmed is parent of Ali



Here we choose parent as the name of a relation; Ahmed and Ali are its arguments. For reasons that will become clear later we write names like ahmed with an initial lower-case letter. In Prolog we can make some statements by using facts. Facts either consist of a particular item or a relation between items. For example, we can represent the fact that it is sunny by writing the program:

sunny.

We can now ask a query of Prolog by asking

?- sunny.

?- is the Prolog prompt. To this query, Prolog will answer yes. **sunny** is true because (from above) Prolog matches it in its database of facts.

Facts have some simple rules of syntax. Facts should always **begin with a lowercase letter** and **end with a full stop**. The facts themselves can consist of any letter or number combination, as well as the underscore _ character. However, names containing the characters -, +, *, /, or other mathematical operators should be avoided.

General Syntax of Fact: predicate (arg1, arg2, arg3, . . . , argN).

Here are some simple facts about an imaginary world. /* and */ are comment delimiters.

- john_is_little. /* john is little */
- raining. /* it is raining */
- john_Forgot_His_Raincoat. /* john forgot his raincoat */

Facts are the declarative sentences which are always true. Facts can represent either relationships between objects or properties of objects. For example:

Facts (Relationships):

Arsalan is father of Saima. **father(arsalan, saima).**

Ali is friend of Farhan. **friend(ali, farhan).**

Farhan can drive car. **drive(farhan, car).**

Ali likes icecream. **likes(ali, icecream).**

Facts (Properties):

Saima is girl. **girl(saima).**

Farhan is man. **man(farhan).**

Farhan can walk. **walk(farhan).**

Ali is happy. **happy(ali).**

b) Rules:

Till now, we added only facts to our program but the real power of Prolog is in rules. While facts state the relation explicitly, rules define the relation in a more general way. Each rule has its head - name of the defined relation, and its body - a real definition of the relation.

A rule can be viewed as an extension of a fact with added conditions that also have to be satisfied for it to be true. It consists of two parts. The first part is similar to a fact (a predicate with arguments). The second part consists of other clauses (facts or rules which are separated by commas) which must all be true for the rule itself to be true. These two parts are separated by “:-” You may interpret this operator as "if" in English.

A rule is an expression that indicates that the truth of a particular fact depends upon one or more other facts. The syntax can be:

predicate1(arg1, arg2, ... argN):- predicate2(arg1, arg2, ... argN), predicate3(arg1, arg2, ... argN),..... , predicateN(arg1, arg2, ... argN).

This can be read as predicate1 is true only if predicate2 to predicateN are true or it can be:

predicate1(arg1, arg2, ... argN):- predicate2(arg1, arg2, ... argN); predicate3(arg1, arg2, ... argN);..... ; predicateN(arg1, arg2, ... argN).

This can be read as predicate1 is true if any one of predicate2 to predicate N is true (as ; represents OR in Prolog).

The left hand side before: - of a rule is restricted to a single, positive, literal, which means it must consist of a positive expression. It cannot be negated and it cannot contain logical connectives.

Here are some examples of rules:

Example 1: Person1 is mother of Person2 if Person1 is female and Person1 is parent of person2.
mother(Person1, Person2) :- female(Person1) , parent(Person1,Person2).

Example 2: Person1 is friend of Person2 if Person1 knows Person2 and Person1 likes Person2.
friend(Person1, Person2) :- knows(Person1, Person2) , likes(Person1, Person2).

Example 3: A mobile is off if it is out of battery or it is shutdown.
off(mobile) :- outofbattery(mobile) ; shutdown(mobile).

Table 8.1: Symbols used in rules

Symbol	Read as
:-	If or implies that
,	And
;	Or
not	Not

c) Questions:

After writing facts and rules we will ask some questions. Those questions will be evaluated from specified facts and rules. For example:

?- **father(shahid, saima)**. Is Shahid father of Saima?

?- **play(ali, cricket)**. Does Ali play cricket?

?- **friend(ali, farhan)**. Is Ali friend of Farhan?

?- **happy(saima)**. Is Saima happy?

III. Operators in Prolog

a) Arithmetic operators:

Prolog can perform all four basic arithmetic operations (addition, subtraction, multiplication, and division) between integral and real values; the type of the result is determined according to the following table.

Table 8.2: Arithmetic operator Rules

Operand 1	Operator	Operand 2	Result
Integral	+, -, *	Integral	Integral
Real	+, -, *	Integral	Real
Integral	+, -, *	Real	Real
Real	+, -, *	Real	Real
Integral or real	/	Integral or real	Real
Integral	mod	Integral	Integral

In case of mixed integral arithmetic, involving both signed and unsigned quantities, the result is signed. The size of the result will be that of the larger of the two operands.

The = operator (predicate) serves two purposes in Prolog. If the left-hand side variable of the expression is bound, it performs comparison else it assigns the value at right hand side to left hand side variable.

b) Relational Operators

Following is the list of relational operators used in prolog:

Table 8.3: Relational operators in Prolog

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equals to
<=	Less than or equals to
=	Equals to
<> or ><	Not equal to

Note that Prolog "=" relation is equality (not assignment); it is the same as the "==" relation in C.

IV. Built-in Predicates in Prolog

In many programming languages there are some built-in functions available. For example in JAVA there Math.sin(), Math.cos(), Math.abs(), System.out.println() etc. are built-in functions. Similarly in PROLOG predicates are like functions and there are some built-in predicates available in PROLOG some of them are explained below.

- **write(value, value1,...):** It prints the values concatenated with each other at output screen. E.g. write ("Hello World", 1, A,B,b) will result in: Hello World1abb (assuming A and B are bound to a and b respectively).
- **nl:** It generates a line feed (new line). The same can be achieved by using \n in write
- **read:-** Used to read values.

Example:

```
-? X is 5+4, Y is 5-4.
X = 9
Y = 1

-? X is sqrt (3), Y is 3^0.5.
X = 1.73205080756888
Y = 1.73205080756888
?- X is 8 mod 3.
X = 2

-? X is 5/3, Y=5//3.
X=1.66666667
Y=1
```

- **Add two numbers**

```
start:- sum,nl.

sum:- write('X= '),read(X),
      write('Y= '),read(Y),
      S is X+Y,
      write('Sum is '),write(S).
```

- **Maximum number**

R is max(3,4).
R = 4.

Making rule for finding the Maximum:

```
max (A, B) :- A>B.
```

Activity- 1:

Convert following sentences in to PROLOG FACTS and specify whether they represent RELATIONSHIP or PROPERTY:

Declarative Sentence	PROLOG FACT	RELATIONSHIP or PROPERTY
SSUET is university		
Faheem likes ice-cream		
Birds can fly		
Switch is off		
Sara is nephew of Saima		

Activity- 2:

Make prolog facts from following sentences and answer the questions by typing queries on prolog.

- Saleem is father of Aslam.
- Dania is mother of Aslam.
- Aslam and Saeed are brothers.
- Faiza is sister of Ali.
- Sana is enemy of Sumra.

Questions

1. List brothers.
2. Who is father of Aslam?
3. Dania is mother of whom?
4. Who is brother of Aslam?
5. Faiza is sister of whom?

Activity- 3:

Create a file and write below fact and execute this using below question.

```
has(jack, apples).
has(ann, plums).
has(dan, money).
fruit(apples).
fruit(plums).
```

Questions

1. List fruits.
2. Which fruit does Jack have?
3. Does Dan have something?

4. Who has apples and who has plums?
5. Does someone have apples and plums?
6. Does Dan have fruits?

Activity- 4:

Write down the PROLOG Rules for the following sentences:

Declarative Sentences	PROLOG RULES
Person1 is sibling of Person2 if Person3 is parent of Person1 and Person3 is parent of Person2.	
Person1 is child of Person2 if Person2 is parent of Person1.	
It will rain today if clouds are heavy and air pressure is low.	

Activity- 5:

A person gets good grades if he studies well and is regular in class. Write program in prolog to check whether a given person gets good grade or not?

Activity- 6:

Here are some declarative sentences convert them into PROLOG facts and answer following questions.

- Kashif likes music.
- Kashif likes cricket.
- Saba likes chips.
- Saba likes mobile.
- Kashif hates cold drinks.
- Saba hates driving.
- Saba hates parties.

Questions

1. Does Saba like chips?
2. Does Kashif hate cricket?
3. Does Kashif like music?
4. What does Kashif like?
5. What does Saba like?
6. Who likes music?

Activity- 7:

There is a cricket team of 5 players; capabilities of each of the player are given in following table.

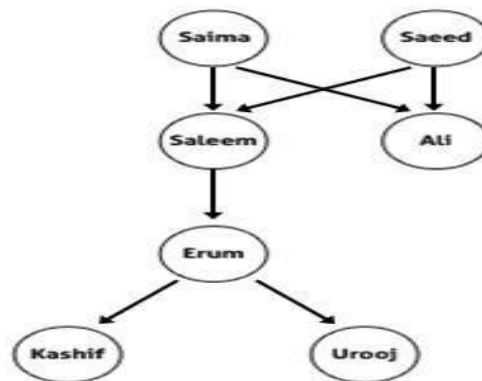
Player Name	Batsman	Bowler	Fielder
Salman	✓		✓
Afridi	✓	✓	✓
Malik	✓	✓	✓
Shoaib		✓	✓
Tanveer		✓	

Write the facts from the above table and create the rule for all Rounder and run following goals in PROLOG.

1. Is Afridi all-rounder?
2. Is Shoaib all-rounder?
3. Who are all-rounders in the team?

Activity- 8:

Case Study: Family Tree: Consider the following family tree shown in figure below

**Questions:**

1. Is Saleem father of Ali?
2. Is Kashif brother of Urooj?
3. Who is father of Kashif?
4. Is Erum mother of Urooj?
5. Who is grandfather of Urooj?
6. Is Salma grandmother of Erum?
7. Saeed is father of whom?
8. Salma is mother of whom?

Activity- 9:

Create selection criteria using rules.

1. starter(soup).
2. starter(salad).
3. main_course(rice).
4. main_course(pizza).
5. main_course(burger).
6. desert(custard).
7. desert(ice_cream).

Create rules for hungry, very_hungry and on_diet.

- For hungry: only starter and main course
- For very_hungry starter, main course and desert
- For on_diet it displays only starters.

Activity- 10:

Write a program that doubles any number that is taken from user as input.

Activity- 11:

Calculate Arithmetic mean of any three numbers.

Activity- 12:

Create a basic calculator prolog program for making basic calculations (add, subtract, multiply, divide, average). Use separate goal statements for making each calculation (with constant numeric values).

Activity- 13:

Make a calculator program by using write() and read() predicates, for taking numeric input from user and displaying text after program execution.